

Mini Search Engine – Information Retrieval System

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

A.Y. 2026-2027

Submitted by
Surapuraju Himasree (2520090181)
Rachapudi Nishanth (2520090141)

Team Number:

Section: 10

Under the guidance of

K. Chandusha

Assistant Professor, Department of Computer Science and Information Technology



Hyderabad-500090, Telangana, India.

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION
TECHNOLOGY



DECLARATION

The Mini Project Report entitled “**Mini Search Engine – Information Retrieval System**” is a record of Bonafide work of **Surapuraju Himasree – 2520090181, Rachapudi Nishanth – 2520090141** submitted in partial fulfillment for the award of B. Tech in Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

Surapuraju Himasree – 2520090181

Rachapudi Nishanth – 2520090141

K L (Deemed to be) University

**DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION
TECHNOLOGY**



CERTIFICATE

This is to certify that the mini project based report entitled “**Mini Search Engine – Information Retrieval System**” is a bonafide work done and submitted by **Surapuraju Himasree – 2520090181, Rachapudi Nishanth – 2520090141** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science and Information Technology, K L (Deemed to be University), during the academic year **2026-2027**.

Signature of the Supervisor

Signature of the HOD

Signature of the External Examiner

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my guide, K. Chandusha, Assistant Professor, Department of Computer Science and Information Technology, which cannot be expressed in words for her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. Khaja Shareef (CSIT)**, Head of the Department for Computer Science and Information Technology for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

Surapuraju Himasree – 2520090181

Rachapudi Nishanth – 2520090141

TABLE OF CONTENTS

S.No.	Contents	Page no.
1.	Abstract	6
2.	Introduction	7
3.	Literature survey	8
4.	Problem Statement and Objectives	9
5.	Hardware and Software requirements	10
6.	Implementation	11
7.	Results & Discussion	13
8.	Conclusion & Future Scope	15
9.	References	16

1.ABSTRACT

The Mini Search Engine – Information Retrieval System is a Java application that searches and retrieves relevant information from a collection of text documents without scanning every document for every query. A breadth-first web crawler collects up to 20 Wikipedia articles (link depth 2), the text is cleaned and tokenised, and an inverted index maps every word to the documents that contain it. Users query the system through a simple browser page that supports exact keyword search, exact phrase search, fuzzy search for misspelled words (using edit distance) and document similarity search (using cosine similarity), while TF-IDF ranking is available through the API. Every result is returned with a snippet and a relevance score, so the most useful documents appear first.

All core algorithms – indexing, edit distance, ranking and similarity – were implemented from scratch using only the Java standard library, and the efficiency of each was analysed using time complexity. Testing on sample documents confirmed the expected output for every search type. The project demonstrates the practical use of hash maps, queues, sets, dynamic programming and vector arithmetic in building a simple yet effective document retrieval system.

Team Name:

Team Logo (if any): None

Team Members:

1. Surapuraju Himasree, 2520090181
2. Rachapudi Nishanth, 2520090141

2. INTRODUCTION

1.1 Background

Digital text collections are growing at a rapid pace. Articles, notes, web pages and reports pile up faster than anyone can read them, and manually scanning every document to find one fact quickly becomes slow and impractical. Large search systems such as Google, Apache Lucene and Elasticsearch solve this problem by organising documents with an index and by ranking the matches so that the most useful results come first.

The Mini Search Engine – Information Retrieval System is a small-scale version of the same idea, written entirely in Java and built from scratch. A web crawler collects pages from Wikipedia, each page is stored as a WebDocument (title, text and URL), the text is cleaned and split into words, and an inverted index maps every word to the documents in which it occurs. Users search through a simple web page that offers keyword, exact-phrase, fuzzy and similarity search, and every result is returned with a snippet and a relevance score.

1.2 Scope

The system covers the complete retrieval pipeline: crawling Wikipedia articles (up to 20 pages and a link depth of 2 from a starting page), cleaning and tokenising the text, building an in-memory inverted index, and answering queries through five retrieval modes – keyword, phrase, fuzzy, TF-IDF ranking and cosine-similarity search. Results are exposed through a small HTTP API and displayed by a browser-based front end. Persistent storage of the index, stemming and stop-word removal, distributed crawling, user accounts and multi-language support are outside the scope of this submission.

1.3 Importance and Target Users

A search engine is one of the most practical applications of data structures and algorithms: hash maps give constant-time lookup, queues drive the crawler, dynamic programming computes edit distance, and vector arithmetic measures similarity. Building these pieces by hand gives hands-on insight into indexing, ranking and string processing. The system serves end users, who type a query, choose a search type and read the ranked results, and students and instructors, who can use the source code as a learning tool.

The rest of this report is organised as follows: Chapter 3 surveys similar systems, Chapter 4 states the problem and objectives, Chapter 5 lists the hardware and software requirements, Chapter 6 describes the implementation, Chapter 7 presents the results, and Chapter 8 gives the conclusion and future scope.

3. LITERATURE SURVEY

The following search systems and crawlers are similar in purpose to this project. Each was studied to understand how real systems organise documents, match queries and rank results; their URLs are listed in the References chapter.

Apache Lucene. An open-source Java library for full-text indexing and search. It is built around an inverted index and offers text analysis, relevance scoring, phrase queries and fuzzy queries based on edit distance. Lucene is the foundation of Solr and Elasticsearch, and it is the closest real-world counterpart of this project, which re-implements the same core ideas from scratch without using any search library.

Elasticsearch. A distributed search and analytics engine built on top of Lucene that is accessed through a REST/JSON API. Documents are indexed on arrival and queries return ranked hits with scores. The `/search` JSON endpoint of this project follows the same request–response idea on a much smaller scale.

Apache Solr. An open-source enterprise search platform, also built on Lucene, that is used through HTTP APIs. It adds features such as faceting, caching and configurable query handling on top of the Lucene index. It shows how an inverted index can be wrapped in a server so that browsers and applications can query it.

Whoosh. A full-text indexing and searching library written in pure Python. It shows that an indexer, a query parser and a ranking function can be implemented without native code, which makes it a good reference for understanding the structure of a search library.

Xapian. An open-source probabilistic information-retrieval library written in C++ with bindings for several other languages. It provides indexing, ranked retrieval and query expansion, and illustrates a ranking model different from the TF-IDF and cosine-similarity measures used here.

Apache Nutch. An extensible, scalable open-source web crawler. It fetches pages, extracts links and hands the content over for indexing. The crawler of this project works on the same principle, but it is single-threaded, limited to 20 Wikipedia pages and uses a simple queue and visited set instead of a distributed fetch cycle.

Meilisearch. An open-source search engine that emphasises fast results and typo tolerance. Its tolerance for misspelled queries is comparable to the fuzzy search of this project, although this project uses a fixed edit-distance threshold of 1 and compares the query with every word.

Typesense. An open-source, typo-tolerant search engine designed for instant search-as-you-type experiences. Like the other engines above, it keeps an in-memory index for speed, which is also the approach taken by this project.

Summary. Production systems are powerful but hide their algorithms inside libraries. This project fills that gap for learning: it implements the inverted index, edit distance, TF-IDF and cosine similarity by hand and analyses the time complexity of each, so that the working of a search engine is fully visible.

4. Problem Statement and Objectives

Problem Statement:

To search a collection of text documents quickly and return the most relevant ones, ranked by relevance, without reading every document manually.

Problem Description:

Given a collection of documents, the system must find which documents contain a word, an exact phrase or a word close to a misspelled query, and which documents are most related to the query, using efficient data structures and algorithms with analysed time complexity.

Objectives:

Project's Purpose:

The project was started to show how a search engine retrieves information from many documents quickly and in order of relevance, and to build the supporting data structures and algorithms from first principles instead of using a search library.

Objectives:

- Retrieve relevant information from a collection of text documents quickly, without scanning every document for every query.
- Organise documents with an inverted index that maps each term to the documents containing it.
- Support multiple search methods: exact keyword search, phrase search, fuzzy search for misspelled words, and document similarity search.
- Rank the retrieved documents by relevance (TF-IDF and cosine similarity) so that the most useful results appear first.
- Implement the core algorithms – indexing, edit distance, ranking and similarity – from scratch instead of using a search library.
- Analyse the efficiency of every algorithm using time complexity.
- Provide a simple, user-friendly web interface on top of a Java backend.

5. HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements:

- Processor: Intel Core i3 (or equivalent) and above
- RAM: 4 GB minimum, 8 GB recommended (the index is held in memory)
- Storage: at least 500 MB of free disk space for documents, index files and project setup
- Display: standard monitor with a minimum resolution of 1366 x 768

Software Requirements:

- Operating system: Windows 10 / 11 (used to develop, build and run the project)
- Java Development Kit (JDK): Java 11 or later, because the crawler uses the `java.net.http HttpClient`. The supplied classes in the bin folder were produced by JDK 25 (class-file version 69), so either run them with the same JDK or recompile the sources with the installed one
- Code editor: Visual Studio Code with the Java extensions
- Web browser: Chrome, Edge or Firefox, to open the front end
- Internet connection while the crawler is running, because it downloads Wikipedia pages
- Version control: Git and a GitHub account
- Languages and libraries: Java standard library only (`java.net.http`, `java.util`, `java.util.regex`, `com.sun.net.httpserver`); HTML, CSS and JavaScript for the front end; no third-party dependencies

6. IMPLEMENTATION

The backend is organised into five modules, each made of small classes with one responsibility. The front end is a single static page (index.html, style.css, script.js) that calls the backend through one HTTP endpoint. The main modules and their key code are given below.

Module 1: Crawler (WebCrawler)

Starts from a seed URL (the Wikipedia article on the Java programming language) and keeps a FIFO queue of (URL, depth) items. For each URL it checks that the URL has not been visited, is a valid en.wikipedia.org article and is within the maximum depth; it then downloads the page with HttpClient, accepts only HTTP 200 responses with a text/html content type, extracts the title and readable text with regular expressions, stores a WebDocument and adds the page's links to the queue. It stops after 20 pages. The logic is summarised in the pseudocode below.

```
queue.add(seedUrl, depth 0)
while queue is not empty and pages collected < 20:
    (url, depth) = queue.remove()
    if url in visited or depth > 2 or not isValidLink(url): continue
    visited.add(url)
    response = HttpClient.send(url)
    if status == 200 and contentType is text/html:
        documents.add(new WebDocument(title, cleanText, url))
        for link in extractLinks(page): queue.add(link, depth + 1)
```

Module 2: Text Processing and Indexing (TextProcessor, InvertedIndex)

Every document is lower-cased, every character other than a–z, 0–9 and space is replaced by a space, and the text is split into words. Each word is then added to the inverted index with its document ID; the posting list is created when a word is seen for the first time and duplicate document IDs are avoided (InvertedIndex.java).

```
word = word.toLowerCase();
if (!index.containsKey(word))
    index.put(word, new ArrayList<Integer>());
ArrayList<Integer> documents = index.get(word);
if (!documents.contains(documentId))
    documents.add(documentId);
```

Module 3: Search (KeywordSearch, PhraseSearch, FuzzySearch)

Keyword search looks the lower-cased query up in the index and maps the IDs back to documents. Phrase search lower-cases each document and checks whether it contains the phrase. Fuzzy search compares every word of every document with the query using edit distance and accepts a document when any word is within the threshold of 1. The dynamic-programming core of the edit-distance routine is shown below (EditDistance.java).

```
int[][] dp = new int[m + 1][n + 1];
for (int i = 0; i <= m; i++) dp[i][0] = i;
for (int j = 0; j <= n; j++) dp[0][j] = j;
for (int i = 1; i <= m; i++) {
    for (int j = 1; j <= n; j++) {
        if (a.charAt(i - 1) == b.charAt(j - 1))
            dp[i][j] = dp[i - 1][j - 1];
        else
            dp[i][j] = 1 + Math.min(dp[i][j - 1],
                                    Math.min(dp[i - 1][j], dp[i - 1][j - 1]));
    }
}
```

```

}
return dp[m][n];

```

Module 4: Ranking and Similarity (Ranker, TFIDF, SimilaritySearch, CosineSim)

The Ranker computes TF-IDF scores for a term and sorts the documents from highest to lowest. SimilaritySearch builds a common vocabulary, converts the query and every document to term-frequency vectors, scores them with cosine similarity and sorts the matches (TFIDF.java and CosineSim.java).

```

TF      = termCount / totalWords
IDF     = Math.log((double) totalDocuments / documentsContainingTerm)
score   = TF * IDF
cosine  = dotProduct / (Math.sqrt(magnitudeA) * Math.sqrt(magnitudeB))

```

Server Module and Front-end Logic (SearchServer, script.js)

SearchServer registers the contexts / and /search, parses the query string, calls SearchEngine, creates a snippet (about 250 characters around the first occurrence of the query) and a score for each result, and returns JSON. The front end calls GET /search?query=<text>&type=<keyword|phrase|fuzzy|rank|similarity> with the fetch API and shows one card per result with its rank, linked title, URL, snippet and relevance percentage. Output is escaped on the server (escapeJson) and in the browser (escapeHtml), so crawled text cannot break the JSON or the page. The response has the following shape:

```

{ "query": "pythn", "type": "fuzzy", "count": 1,
  "results": [ { "title": "...", "url": "...",
                 "snippet": "...", "score": 1.0000 } ] }

```

Time Complexity of the Main Operations

N = number of documents, T = total number of words, V = vocabulary size, k = number of matches, m = query length, n = length of a word.

Operation	How it works	Approximate time
Tokenising a text of length L	lower-case, regex clean-up, split	$O(L)$
Building the inverted index	one HashMap insert per word; duplicate check in the posting list	$O(T)$ average; $O(T \cdot N)$ worst case
Keyword search	HashMap lookup + collect k documents	$O(1)$ average + $O(k)$
Phrase search	String.contains on every document	$O(\text{total text length})$
Edit distance	dynamic-programming table	$O(m \cdot n)$ time and space
Fuzzy search	edit distance against every word of every document	$O(T \cdot m \cdot n)$
TF-IDF ranking	count term, compute IDF, bubble sort k results	$O(T) + O(k^2)$
Cosine similarity of two vectors	dot product and magnitudes	$O(V)$
Similarity search	vocabulary via ArrayList.contains, N vectors, bubble sort	$O(T \cdot V) + O(N \cdot V) + O(k^2)$
Crawling	BFS over at most 20 pages	$O(P)$ pages; network time dominates

Table 6.1: Time complexity of the main operations

7. RESULTS & DISCUSSION

The system was tested on three sample documents (java.txt, python.txt and web.txt), which produce an inverted index of 27 distinct terms, and every search type was queried exactly as SearchServer does. Table 7.1 shows representative queries and their results.

Search type	Query	Result (document, score)
Keyword	java	Java, Web Development (score 1.0)
Phrase	programming language	Java, Python (score 1.0)
Fuzzy	pythn	Python (score 1.0)
Rank (TF-IDF)	java	Java 0.0760, Web Development 0.0253
Similarity	java backend	Java 0.5547, Web Development 0.3162

Table 7.1: Sample queries and results

Discussion

- **Keyword search** is case-insensitive (“JAVA” gives the same result as “java”) and returns nothing for an absent word such as “zebra” or for a two-word query, because keyword mode matches a single term.
- **Phrase search** finds “programming language” in the Java and Python documents, while “machine learning” is found only in the Python document.
- **Fuzzy search** recovers from spelling mistakes: “pythn” finds the Python document and “jav” finds the documents containing “java”.
- **TF-IDF ranking** places the Java document (0.0760) before Web Development (0.0253) for “java”, because the word is more frequent in it.
- **Similarity search** scores the Java document 0.5547 and Web Development 0.3162 for “java backend”, and leaves out the Python document, which shares no words with the query.

Test Cases

Of the 21 test cases, cases 1–18 were run on the sample documents and all passed; cases 19–21 need a live internet connection, a running server and a browser, so their results are recorded during the demonstration.

#	Test case	Expected result	Actual
1	Keyword “java”	Java and Web Development documents	Pass
2	Keyword “JAVA” (upper case)	Same two documents (case-insensitive)	Pass
3	Keyword “zebra” (not present)	0 results	Pass
4	Empty query	Empty result list	Pass
5	Keyword “java backend” (two words)	0 results – keyword mode matches a single term	Pass
6	Phrase “programming language”	Java and Python documents	Pass
7	Phrase “machine learning”	Python document only	Pass

#	Test case	Expected result	Actual
8	Fuzzy “pythn”	Python document	Pass
9	Fuzzy “jav”	Java and Web Development documents	Pass
10	Rank “java”	Java (0.0760) before Web Development (0.0253)	Pass
11	Rank “programming”	Java (0.0507) before Python (0.0239)	Pass
12	Similarity “java backend”	Java (0.5547), Web Development (0.3162); Python excluded	Pass
13	Unknown search type	Falls back to keyword search	Pass
14	Edit distance “kitten” → “sitting”; “java” → “jav”	3 and 1	Pass
15	Cosine similarity of identical and orthogonal vectors	1.0 and 0.0	Pass
16	DocumentSimilarity of Java and Python documents	Value between 0 and 1 (≈ 0.458)	Pass
17	TextProcessor on “Hello, World! Java-17 rocks...”	hello, world, java, 17, rocks	Pass
18	Size of inverted index for the three sample documents	27 distinct terms	Pass
19	Crawler with 20 pages, depth 2 from the Java article	Up to 20 valid Wikipedia pages collected	[Record after demo]
20	GET /search over HTTP for each search type	JSON with query, type, count and results	[Record after demo]
21	Front end with server stopped	“Could not connect to the search server” message	[Record after demo]

Table 7.2: Test cases and results

Application Screenshots

Figure 7.1 – Home page: [Insert screenshot here.] The page shows the search box, the Search button and the four search-type buttons: Keyword, Exact Phrase, Fuzzy and Similarity.

Figure 7.2 – Keyword and exact phrase search results: [Insert screenshot here.] Each result card shows the rank number, linked title, URL, snippet and relevance percentage for the matching documents.

Figure 7.3 – Fuzzy search with a misspelled word: [Insert screenshot here.] A misspelled query such as “pythn” still returns the Python document because its edit distance to “python” is 1.

Figure 7.4 – Similarity search results: [Insert screenshot here.] Documents are listed from the highest to the lowest cosine-similarity score, shown as a relevance percentage.

Figure 7.5 – “No results found” page and server-not-running error: [Insert screenshot here.] Clear messages are shown for an unmatched query and when SearchServer is not running on port 8080.

Figure 7.6 – Server console: [Insert screenshot here.] The console prints the crawler progress (URL, HTTP status, title, pages collected), then “Search index created.” and “Mini Search Engine Server Started”.

8. CONCLUSION & FUTURE SCOPE

Conclusion

This project set out to make searching a collection of text documents fast and easy, and the objectives were met. A Java web crawler collects Wikipedia pages, an inverted index organises their words, and five retrieval algorithms – keyword, phrase, fuzzy, TF-IDF ranking and cosine similarity – answer queries submitted from a simple web page. Results are returned with a snippet and a relevance score through a small JSON API.

All of the core algorithms were written from scratch using only the Java standard library, and their efficiency was analysed using time complexity. Testing on the sample documents confirmed the expected results for each search type, for the edit-distance and cosine-similarity routines, and for the index. The project also gave practical experience in choosing data structures for a task, breadth-first crawling, string and HTML processing with regular expressions, building an HTTP server and connecting a JavaScript front end to a backend.

Limitations

- Keyword search and TF-IDF ranking handle a single term only; stop-word removal, stemming and word positions are not implemented.
- Fuzzy search uses a fixed distance of 1, scans every word of every document and returns unranked results.
- Bubble sort is used for ranking ($O(k^2)$); the index lives only in memory and is rebuilt by crawling at every start-up.

Future Scope

Near term: add stop-word removal and stemming so that “run”, “runs” and “running” match each other; support multi-word and Boolean queries (AND, OR, NOT) with TF-IDF over every query term; add a button for TF-IDF ranking in the web page, result pagination, highlighting of matched words, and a “did you mean ...?” correction based on edit distance.

Medium term: replace bubble sort with merge sort or a heap that keeps only the top-k results ($O(k \log k)$); use a hash map for the vocabulary and sparse vectors to speed up similarity search; build a positional inverted index so that phrase search can use the index; save the index to disk so that it need not be rebuilt after every restart.

Long term: use a trie for prefix search and a BK-tree for fuzzy matching without comparing every word; make the crawler polite and faster (respect robots.txt, add delays, fetch pages in parallel threads); add automated tests (JUnit), a build tool such as Maven or Gradle, and a Docker image for one-command start-up.

9. REFERENCES

1. C. D. Manning, P. Raghavan and H. Schütze, Introduction to Information Retrieval, Cambridge University Press, 2008. <https://nlp.stanford.edu/IR-book/>
2. T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, MIT Press.
3. Java Platform, Standard Edition – Documentation (java.net.http, java.util, com.sun.net.httpserver). <https://docs.oracle.com/en/java/javase/>
4. Apache Lucene – Documentation. <https://lucene.apache.org>
5. Elasticsearch – Documentation. <https://www.elastic.co/elasticsearch>
6. Apache Solr. <https://solr.apache.org>
7. Whoosh – Documentation. <https://whoosh.readthedocs.io>
8. Xapian – Open Source Search Engine Library. <https://xapian.org>
9. Apache Nutch – Web Crawler. <https://nutch.apache.org>
10. Meilisearch. <https://www.meilisearch.com>
11. Typesense. <https://typesense.org>
12. Levenshtein distance (edit distance) – Wikipedia. https://en.wikipedia.org/wiki/Levenshtein_distance
13. tf-idf – Wikipedia. <https://en.wikipedia.org/wiki/Tf%E2%80%93idf>
14. Cosine similarity – Wikipedia. https://en.wikipedia.org/wiki/Cosine_similarity
15. MDN Web Docs – JavaScript, the Fetch API and DOM reference. <https://developer.mozilla.org>
16. Visual Studio Code – Documentation. <https://code.visualstudio.com/docs>
17. GitHub Docs. <https://docs.github.com>